Métodos en objetos

Para definir un método en un objeto basta con asignarle una **función** a una **propiedad** del mismo. Podemos hacerlo de varias formas:

```
function f(){

    console.log("Hola");

};

let obj = {};

obj.metodo = f;

obj.metodo();

Hola
```

También

```
let obj = {

    metodo : function(){

        console.log("Hola");

    }

}
```

Y, más compacto:

```
obj = {

     metodo(){

        console.log("Hola");
```

```
    }

}
```

Contexto de ejecución de un método (this)

La palaba `this` en `el context` en los navegadores hace referencia **al objeto** `window`.

La palabra `this`, cuando se invoca en una función, referencia **al objeto sobre el que se ejecuta la función**.

NOTA: Hay que recordar que, si ejecutamos una función en el nivel global, el método se ejecuta sobre el objeto `window` por lo que, en ese caso, dentro de la función `this === window`:

```
function quienSoy(){

    console.log(this);
```

```
> prod2=200

console.log("Producto " + prod2);

>Producto 200

let prod3 = {

    nombre: 'Prod3',

    precio: 300,

    toString: function() { return this.nombre;}

}

console.log("Producto " + prod3);

> Producto Prod3

console.log(prod3 * 2);

> NaN
```

Constructores

Los **constructores** en Javascript son **funciones normales** que son invocadas a través del **operador** `new`. Por convención se **recomienda** que sus nombres comiencen por una letra mayúscula.

Cuando se llama a una función con el operador `new` se realizan las siguientes operaciones:

- Se **crea un nuevo objeto vacío** (`{}`)
- Se **asigna** dicho objeto a `this`
- Se **ejecuta** la función
- Se **retorna** el objeto `this` creado

```
function Prod(nombre, precio){

       this.nombre    =    nombre;

      this.precio    =    precio;

}

prod1 = new Prod("Prod1",100);

console.log(`${prod1.nombre} : ${prod1.precio}`);


>Prod1 : 100
```

NOTA: La función constructora normalmente no se pone Return , porque te devuelve el objeto que está construyendo. No obstante, **puede retornar un valor explícitamente**.

NOTA: Si un constructor **no tiene argumentos**, se **pueden omitir los paréntesis al emplear el operador new** (`new Constructor`)

```
Por ejemplo :

let producto = new Prod() o

let producto = new Prod

Siempre y cuando Prod no tenga argumentos
```